FPGA 与 Zynq 核心认知重构教程

从“写软件指令”到“搭并行电路”的完整脉络与实战闭环

教程设计导读：

许多具备单片机（STM32）或 C/Python 基础的开发者在初学 FPGA 时，最容易陷入**“拿串行软件思维硬套并行硬件电路”**的困境。本教程打破传统教材枯燥的语法罗列，严格按照**“最不可绕过 → 最核心基石 → 实战全流程闭环 → 工业工程规范”**的人类认知规律编排：

- **第一篇（思维破壁）**：彻底厘清 PL/PS/PC 边界、硬件并行本质、时钟节拍与 LUT 物理映射。
- **第二篇（最小原语）**：掌握 Verilog 语法树、方向与类型正交性、单驱动扇出、阻塞与非阻塞赋值、位宽并行运算。
- **第三篇（空间架构）**：理解模块定义与例化、层次包含关系，以及叶子/中间/顶层三级模块分工。
- **第四篇（虚实映射）**：打通 XDC 物理引脚约束与时钟基准，理解逻辑端口如何映射到芯片物理焊球。
- **第五篇（实战闭环）**：**重点补全写完 `.v` 和 `.xdc` 之后的必经之路**——Testbench 仿真、综合与布局布线、建立/保持时间与 WNS 时序收敛、片上逻辑分析仪（ILA）在线抓波。
- **第六篇（构建烧录）**：剖析 `.bit` 协议流、JTAG 位序，以及 PL/PS/Zynq 整包从源码到上电的全链路。
- **第七篇（工程规范）**：建立工业级目录隔离（`fpga/ps/host`）、生成物管理（`build` vs `artifacts`）与自动化脚本体系。

作者：刘昊岳

2026 年 10 月 15 日

目录

1 第一篇：思维破壁——从“写软件指令”到“搭并行电路” 3
1.1 三角色边界：PL、PS、PC 到底谁是谁? . . . 3
1.2 时序、并行与“数节拍”之谜 . . . 4
1.2.1 疑惑一：FPGA 为什么不需要在代码里自己“数节拍”? . . . 4
1.2.2 疑惑二：一个模块需要两个模块的输入，怎么“等”两个都到齐? . . . 4
1.3 底层物理映射：数字逻辑门、LUT 与编译器优化 . . . 5
1.3.1 疑惑一：FPGA 内部真的是由“与门、或门、非门”搭出来的吗? . . . 5
1.3.2 疑惑二：硬件编译器（综合器）会做数字逻辑优化吗? . . . 6

2 第二篇：最小硬件原语——Verilog 核心语法与数据表示 6
2.1 自由格式、词法规则与代数语法树 . . . 6
2.2 模块声明拆解：方向与类型的正交设计 . . . 7
2.3 驱动模型与赋值铁律：`wire` 扇出、`assign` 与 `always` . . . 8
2.3.1 疑惑一：`wire` 能连接多个点吗? 如果多个模块同时往一根线上发数据会怎样? . . . 8
2.3.2 疑惑二：`assign` 和 `always` 在芯片底层器件上有区别吗? `q <= d;` 有什么硬性要求? . . . 8
2.4 位宽、扩展、切片与并行位运算 . . . 10
2.5 Verilog 高频关键字分类速查表 . . . 10

3 第三篇：空间架构——模块化层次与顶层设计 10
3.1 模块定义与例化：是面向对象里的“类与继承”吗? . . . 12
3.2 三级模块分工：叶子模块、中间模块与顶层（Top） . . . 12
3.3 端口映射的精确语义：`.clk(sys_clk)` 到底谁连谁? . . . 14

4 第四篇：虚实映射——XDC 物理约束与时序基准 14
4.1 三级物理传导链路：XDC 绑定的到底是谁? . . . 14
4.2 逐词拆解 XDC 语法：`[get_ports]` 与 `create_clock` . . . 15

5 第五篇：【核心补充】写完 `.v` 和 `.xdc` 之后——从仿真验证到板级闭环 17
5.1 第一关：行为级仿真（Testbench）——为什么绝不能跳过仿真直接上板? . . . 17
5.1.1 疑惑一：Testbench 也是 `.v` 文件，它和我们写的电路模块有什么区别? . . . 17
5.1.2 疑惑二：怎么写一个最简万用 Testbench? 怎么看懂仿真波形里的红线和蓝线? . . . 18
5.2 第二关：综合（Synthesis）与实现（Implementation）——工具在后台干了什么? 19
5.3 第三关：看懂时序报告与 WNS——怎么知道电路在高速下能不能跑稳? . . . 20
5.4 第四关：片上逻辑分析仪（ILA）——上板后内部信号不对怎么抓? . . . 21

6 第六篇：构建链路与底层烧录机制 21

6.1 为什么只写 .v 不能像软件一样直接“一键生成可执行文件”? 21

6.2 三条核心构建链路与烧录产物矩阵 22

6.3 .bit 比特流解剖学、JTAG 位序与主被动加载模型 22

7 第七篇：工业级工程规范与目录架构 24

7.1 标准工程目录树与三端职责隔离 24

7.2 生成物生命周期：build/（厨房半成品）vs artifacts/（留样成品） 25

7.3 自动化构建体系：scripts/、Makefile 与词源辨析 25

1 第一篇：思维破壁——从“写软件指令”到“搭并行电路”

在动手写第一行 Verilog 代码之前，有一道**绝对不可绕过**的认知门槛：你写的不是给 CPU 一条条读取执行的“程序指令”，而是一张用来在硅片上铺设真实电路的“施工图纸”。

1.1 三角色边界：PL、PS、PC 到底谁是谁？

在 Zynq 等异构 SoC 开发中，初学者最容易把 **PL**、**PS** 和 **PC** 混淆，尤其是 PS 和 PC 仅一字之差，但物理位置和职责完全不同。

【直觉卡点】PS、PL、PC 傻傻分不清楚

“工程里既有 Verilog，又有 C 语言，还有 Python，它们分别跑在哪里？PS 和 PC 有什么区别？”

表 1: PL、PS、PC 三大角色核心边界对比

缩写	英文全称	物理位置	开发语言	编译产物	核心本质与职责
PL	Programmable Logic	Zynq 芯片内部	Verilog / SV	`.bit` / `.bin`	**可编程逻辑电路**：你在芯片里搭的纯硬件数字电路（高速采集、并行滤波等）
PS	Processing System	Zynq 芯片内部	C / C++	`.elf` / `.bin`	**片上处理系统**：同一颗芯片里硬核集成的 ARM 处理器（跑裸机控制或嵌入式 Linux）
PC	Personal Computer	开发板芯片外部	Python / C#	不烧录进板子	**外部上位机主机**：你桌面上那台电脑，通过网口/串口/USB 发送指令、接收波形数据

【核心结论】一句话记住三角色

PL 是芯片里的定制电路，PS 是同一颗芯片里的 ARM 小电脑，PC 是芯片外面你桌上的电脑主机。

1.2 时序、并行与“数节拍”之谜

习惯了单片机（如 STM32）的开发者，脑子里总有一个“程序计数器（PC 指针）”在从上往下逐行跑代码。看到 Verilog 代码时，往往会产生极大的困惑。

1.2.1 疑惑一：FPGA 为什么不需要在代码里自己“数节拍”？

【直觉卡点】既然是数字电路，为什么代码里没看到数节拍？

“我们写的 Verilog 代码貌似没有显式地去数节拍啊？既然是同步数字电路，难道不需要自己在代码里一步步数拍子推进吗?”

【硬核拆解】硬件时钟沿天然就是节拍器

1. **CPU 靠程序计数器推进**：CPU 内部有指令译码器和程序计数器（PC），靠硬件自动取指来一步步推进代码行。
2. **FPGA 没有 CPU 和指令，但有全局时钟与触发器（Flip-Flop）**：在 FPGA 中，**物理时钟的上升沿（`posedge clk`）本身就是全芯片统一的节拍器**。
3. **自动同步更新**：当你写下 `always @(posedge clk)` 时，就代表电路里的触发器在每一个时钟上升沿到来瞬间，同时锁存一次输入并更新输出。不需要你写代码去驱动节拍，**晶振产生的时钟脉冲正在替你数每一拍**。
4. **何时才写显式计数器？**只有当你需要“每隔 1000 拍翻转一次 LED”或“严格等待 16 拍完成一次 ADC 采样”时，才会在代码里定义一个 `cnt` 寄存器去显式计数。

1.2.2 疑惑二：一个模块需要两个模块的输入，怎么“等”两个都到齐？

【直觉卡点】用串行等待思维看并行硬件

“假设模块 C 需要模块 A 和模块 B 的输出作为输入，怎么才能等到两个都到齐呢？会不会提前只提取了一个？还是说数据到了就在寄存器置 1，等两个都置 1（与关系），才进入下一节拍？而且时钟允许这样干等吗？FPGA 不是为了高效吗?”

【硬核拆解】硬件里没有“串行等待”，只有“持续并行 + 握手同步”

1. **物理层面上不存在“先执行 A 再执行 B”**：在硅片上，模块 A、模块 B、模块 C 是三块同时通电的物理电路。每个时钟沿，大家同时工作；如果是纯组合逻辑 `assign c = a & b;`，电信号一直连着，`a` 或 `b` 的电平一变，经过纳秒级门延迟，`c` 立刻跟着变。

2. **当两路数据计算耗时不同（如 A 算 3 拍、B 算 8 拍）怎么对齐？**这时正是你直觉猜到的机制——**显式设计握手信号（Valid / Ready 协议）**：
   - 模块 A 算完后，拉高 `a_valid = 1` 并保持数据不变；
   - 模块 B 算完后，拉高 `b_valid = 1`；
   - 模块 C 的使能条件写为 `if (a_valid && b_valid)`，只有两者相与为真时，才在下一个时钟沿把两路数据同时吞进去处理。
3. **为什么“等待”不会拖垮 FPGA 的高效性？**时钟既不快也不慢，它是固定频率的节拍（如 100MHz 即每 10ns 一拍）。FPGA 的高效来自**空间并行与流水线（Pipeline）**——当模块 C 在等第 1 组数据对齐时，前级电路已经在流水线式地吞入第 2、第 3 组数据了，全芯片成百上千个算子在同一拍内同时运转。

1.3 底层物理映射：数字逻辑门、LUT 与编译器优化

1.3.1 疑惑一：FPGA 内部真的是由“与门、或门、非门”搭出来的吗？

【直觉卡点】写下 `a & b`，芯片里真的长出一个与门吗？

“数字逻辑的基本门是与、或、非。那我们在 Verilog 里写逻辑运算，FPGA 内部真的全是一个个与或非门吗?”

【硬核拆解】FPGA 用查找表（LUT）模拟一切布尔逻辑

- **理论层是与或非**：布尔代数的基本单元确实是与（AND）、或（OR）、非（NOT）、异或（XOR）等逻辑门。
- **物理层是 LUT（Look-Up Table，查找表）**：如果芯片出厂时把与门、或门焊死，就没法灵活变成任意电路了。因此，现代 FPGA 内部使用的是 **LUT（本质是一小块可读写的真值表配置内存，如 6 输入 LUT 有 64 项结果）**。
- **怎么实现的？**当你写下 `assign y = (a & b) | c;` 时，综合器并不会在硅片上临时造出一个与门和一个或门，而是把该表达式对应的**真值表（0/1 结果序列）**提前算好，烧录进 LUT 的存储单元里。工作时，输入信号 `a`，`b`，`c` 相当于查表地址，直接把对应的结果读出来——**任何一个 N 输入的复杂布尔函数，在硬件上都只需查一次表即可完成！**

1.3.2 疑惑二：硬件编译器（综合器）会做数字逻辑优化吗？

【核心结论】综合器的优化极其激进，且严格保证功能等价

Vivado 等综合工具在把代码变成电路时，会自动执行全套数字逻辑优化：

1. **常量传播（Constant Propagation）**：如 `a & 1'b1` 直接化简为连线 `a`；`a & 1'b0` 直接接地 `1'b0`。
2. **死代码消除（Dead Code Elimination）**：定义了但最终没有影响到任何顶层输出引脚的寄存器和逻辑，会被直接剪除（Trimmed），不占哪怕一个 LUT。
3. **逻辑化简与公共子表达式提取**：自动完成类似卡诺图化简的工作，将多处重复的计算合并为一个硬件单元。
4. **寄存器重定时（Retiming）**：在不改变外部节拍行为的前提下，自动把组合逻辑中间的触发器往前或往后挪动，以平衡两级之间的延迟，让电路跑上更高主频。

—

2 第二篇：最小硬件原语——Verilog 核心语法与数据表示

建立了“搭电路”的心智模型后，我们来拆解描述电路的语言工具——Verilog 的最小核心语法。

2.1 自由格式、词法规则与代数语法树

【直觉卡点】空格、缩进、分号和逗号到底哪些是硬性规定？

“Verilog 对空格和缩进有要求吗？`top` 能和后面的括号分开吗？它是不是像 C 语言一样逐个关键字读取？分号 `;` 和逗号 `,` 的本质区别是什么？这里的语法树怎么理解？”

【硬核拆解】自由格式语言的词法与语法树解析

1. **缩进与换行完全自由**：Verilog 不像 Python 那样靠缩进划分代码块，而是靠 `begin ... end`（相当于 C 语言的花括号 `{...}`）界定范围。因此，`module top (...)` 中 `top` 和左括号之间加多少空格或换行都合法。
2. **哪些地方绝对不能乱拆？（词法切 Token 规则）**
   - **关键字与标识符之间必须有空白符**：`module top` 不能连写成 `moduletop`，否则词法分析器会把它当成一个未知变量名。
   - **多字符运算符与数字字面量不可拆**：非阻塞赋值 `<=` 中间不能加空格写成 `< =`；位宽字面量 `8'hFF` 中间不可加空格。
3. **分号 `;` vs 逗号 `,`**：
   - **分号 `;` 是完整语句的终止符**（告诉解析器一条声明或赋值语句彻底结束了）。

- **逗号 ，是同级元素的分隔符**（如端口列表 `(input a, input b)` 中，列表还没结束，最后一个端口后面绝不能加逗号）。

4. **表达式的代数语法树（AST）**：编译器读入 `assign y = (a & b) | c;` 时，并非从左到右盲算，而是按运算符优先级（括号 `()` 最高，三目运算符 `?:` 最低）构建出一棵树——最底层叶子节点是信号 `a` 和 `b`，向上汇聚成 `&` 节点，再与 `c` 汇聚成 `|` 节点，最终驱动根节点 `y`。这棵语法树直接对应硬件上逻辑门的级联拓扑！

2.2 模块声明拆解：方向与类型的正交设计

看一段最基础的模块声明代码：

```
module top (
    input  wire sys_clk,    // 输入时钟线
    input  wire rst_n,      // 输入复位线（低有效）
    output reg  led         // 输出寄存器驱动的 LED
);
```

【直觉卡点】只有 `wire` 和 `reg` 吗？和 `input/output` 是固定搭配吗？为什么不融合？

“Verilog 里只有 `wire` 和 `reg` 两种类型吗？它们和 `input`、`output` 是固定绑定吗？为什么不干脆融合成一个关键字？”

【硬核拆解】“信号方向”与“硬件属性”是两个正交维度

在模块端口声明中，每个端口同时具备两个互不相同的物理属性，因此不能盲目融合：

1. **第一维：端口方向（Direction）**——只有三种：`input`（输入）、`output`（输出）、`inout`（双向三态）。它决定信号跨模块边界时的流向。

2. **第二维：数据类型（Data Type）**——在可综合 RTL 电路里，核心就是 **`wire`（物理金属连线，无记忆能力，驱动源撤掉信号就没了）**和 **`reg`（过程赋值变量，在时序逻辑中综合为带记忆能力的触发器 Flip-Flop）**。此外还有仿真或辅助计算用的 `integer`、`real` 等。

3. **两者的组合规则（非死板绑定）**：

   - **输入端口（`input`）**：外界驱动模块内部，模块内部只负责用导线接着，因此 `input` 在模块内**只能是 `wire`**（省略不写也默认是 `wire`）。
   - **输出端口（`output`）**：如果直接由组合逻辑 `assign` 驱动，就是 `output wire`；如果在 `always` 块里被时序触发器驱动，就声明为 `output reg`。

4. **现代演进（SystemVerilog 的 `logic`）**：早期 Verilog 强制区分 `wire` 和 `reg` 确实常让新手困惑。因此在 SystemVerilog（`.sv`）中引入了 `logic` 类型，统一替代了绝大部分单驱动场景下的 `wire` 和 `reg`（综合器会自动根据上下文推断是连线还是触发器），但**多驱动总线和双向端口 `inout` 依然必须显式使用 `wire`**。

2.3 驱动模型与赋值铁律：wire 扇出、assign 与 always

2.3.1 疑惑一：wire 能连接多个点吗？如果多个模块同时往一根线上发数据会怎样？

【直觉卡点】把数字电路当成了可以随意汇合的水管

“一根 wire 只能点对点连接吗？能不能一对多？如果一个模块写了发，另一个模块也写了发，几个模块混着连在同一根导通的线上，怎么保证它是单向不冲突的？”

【硬核拆解】驱动-负载模型：允许“一源对多负载”，严禁“多源驱动一线”

数字电路绝不是可以随意将两股水流汇进同一根主管道的水管系统，它遵循严格的**驱动源（Driver）→ 负载（Load）**电压模型：

- **允许一对多（Fan-out，扇出）**：一根 wire 只能有**唯一一个驱动源**，但可以像树枝分叉一样同时连接到 10 个甚至 100 个模块的输入端（负载）。驱动源输出高电平 3.3V，所有连在这根线上的输入端就同时读到 1。
- **严禁多对一（Multi-driven，多驱动冲突）**：如果模块 A 和模块 B 的输出端直接绑在同一根 wire 上，当 A 输出高电平（1，接电源）而 B 输出低电平（0，接地）时，电源和地直接通过这根线**物理短路打架**！在仿真中会直接变成代表冲突的红线（X 未知态），在 Vivado 综合时会直接报致命错误 [Synth 8-6859] multi-driven net。
- **多路信号想走同一条通道怎么办？**必须显式加入**多路选择器（MUX，用 case 或 ?: 选择当前听谁的）**，或者使用带高阻态（1'bz）的片外三态总线。

2.3.2 疑惑二：assign 和 always 在芯片底层器件上有区别吗？q <= d; 有什么硬性要求？

【核心结论】关于 assign、always 与 q <= d; 的三条铁律

1. **器件本质相同，表达能力不同**：如果用 assign y = a & b; 和 always @(*) y = a & b; 描述同一个与门，综合出来的硬件 LUT 完全一模一样。区别在于 always 里能写 if/case 和时序触发器，而 assign 能简明地直连导线和写三态门。
2. **q <= d; 的类型铁律**：左值 q 处于 always 块内被赋值，语法强制要求**必须是 reg（或 logic）**；右值 d 只是提供数值，是 wire 还是 reg 完全随意。
3. **敏感列表进化**：always @(a, b) 与旧式 always @(a or b) 等价，表示 a 或 b 变化时重新计算；工程中统一写 **always @(*)**，让编译器自动把右值所有信号纳入敏感列表，杜绝漏写信号导致仿真与实际电路不一致（锁存器 Latch 隐患）。

表 2: assign 与 always 块及赋值方式完整对比

语法结构		被赋值左值要求	右值（数据源）要求	对应的底层硬件与适用场景
`assign y = a & b;`		必须是 wire	wire 或 reg 均可	**纯组合逻辑(LUT/连线)**：也可用于描述三态门 `assign io = en ? dout : 1'bz;`
`always @(*)`	内部用阻塞赋值 =	必须是 reg (虽叫 reg，但此处综合出来仍是连线/LUT，不含触发器!)	wire 或 reg 均可	**复杂组合逻辑 (LUT/-MUX)**:支持写 if-else、case 分支;注意敏感列表推荐写 @(*) 自动防漏
`always @(posedge clk)`	内部用非阻塞赋值 <=	必须是 reg (此处真正综合为硬件触发器 Flip-Flop)	wire 或 reg 均可	**同步时序逻辑（触发器 FF）**：在时钟沿瞬间并行更新状态，如 `q <= d;`

2.4 位宽、扩展、切片与并行位运算

【直觉卡点】多位寄存器怎么取其中几位？位数不一致赋值怎么扩？8 位相与是循环算的吗？

“一根 wire 只能连 1 位吗？怎么从多位寄存器里固定取出几位？如果 reg [7:0] q; wire [3:0] d; q <= d; 位宽不一致，高位是怎么扩展的？还有，8 位寄存器做 a & b 逻辑运算时，硬件是怎么算的?”

【硬核拆解】总线切片、扩展规则与空间并行展开

1. **wire 与 reg 均可定义为多位总线（Bus）：**

```
wire [7:0] data_bus;           // 8 根并排走的物理导线，编号 7（最高位 MSB）到 0（最低位 LSB）
wire [3:0] low_nibble  = data_bus[3:0];          // 切片：固定取出低 4 位
wire [7:0] packed_byte = {4'b0000, low_nibble}; // 拼接符：高 4 位显式补 0，拼成 8 位
wire [7:0] all_ones    = {8{1'b1}};              // 复制符：把 1'b1 复制 8 份，即 8'b1111_1111
```

2. **位宽不匹配时的隐式扩展与截断规则：**

   - **左宽右窄（如 8 位 ← 4 位）**：若右值是普通无符号数（默认），**高位自动补 0**；若右值声明了 signed（有符号补码），则**高位自动复制符号位（Sign Extension）**以保持正负数值不变。
   - **左窄右宽（如 4 位 ← 8 位）**：直接**截断丢弃高 4 位**，只保留低 4 位。
   - **工程铁律**：永远不要依赖隐式扩展或截断！务必使用拼接符写成 q <= {4'd0, d};，消除编译器的位宽不匹配 Warning。

3. **多位逻辑运算是“空间并排复制硬件”，绝非时间上的循环**：当你写下 8 位运算 assign y = a & b; 时，综合器会在硅片上直接铺设 **8 个并排独立的与门**，第 0 位与第 0 位相与，第 7 位与第 7 位相与——8 位结果在同一纳秒内同时算出！同理，加法器 a + b 和乘法器 a * b 也是直接综合成硬件全加器链（Carry Chain）或 DSP 硬件乘法单元，一拍出结果。

2.5 Verilog 高频关键字分类速查表

—

3 第三篇：空间架构——模块化层次与顶层设计

单块电路写好后，如何像造大楼一样把成百上千个模块组织起来？这就涉及到 Verilog 的层次化设计（Hierarchy）。

表 3: Verilog (.v) 核心关键字分类全景图

功能分类	核心关键字	工程用途说明
模块与端口	`module`, `endmodule`, `input`, `output`, `inout`	定义电路边界与对外进出端口
参数常量	`parameter`, `localparam`	`parameter` 可在例化时从外部改写配置; `localparam` 为内部死常量
信号类型	`wire`, `reg`, `signed`, `integer`	连线、过程寄存器、有符号补码修饰、循环计数整型
并发与过程	`assign`, `always`, `posedge`, `negedge`, `begin`, `end`	连续赋值、时序/组合过程块、上升沿/下降沿触发、代码块界定
分支与循环	`if`, `else`, `case`, `endcase`, `default`, `for`	优先选择器、多路并行选择器（MUX）、空间复制展开循环
批量生成	`generate`, `endgenerate`, `genvar`	批量例化多个相同子模块或根据参数条件裁剪电路
仅限仿真	`initial`, `$display`, `$finish`, `$readmemh`, `` `timescale ``	**不可综合成硬件!** 仅在 Testbench 仿真时初始化信号、打印日志或读测试文件

3.1 模块定义与例化：是面向对象里的“类与继承”吗？

【直觉卡点】模块和实例怎么理解？父子模块是类继承关系吗？

“模块不能直接调用，还要实例化？这像软件里先写一个类（Class）再创建对象吗？还有，父模块和子模块的关系，难道是面向对象里的父类与子类继承关系？”

【硬核拆解】例化是“按图纸铺设独立电路”，父子是“空间包裹”而非“类继承”

1. **模块定义（Module Definition）= 电路图纸；实例化（Instantiation）= 照着图纸焊一块真实芯片**：
   - 在软件里调函数 `func()`，是 CPU 跳过去跑同一段内存代码，跑完再跳回来（分时复用）。
   - 在硬件里，你不能“调用”一个模块。你写下 `module adc_in(...);` 只不过画了一张图纸，硅片上什么都还没有。只有当你写下 `adc_in u_adc0(...);` 和 `adc_in u_adc1(...);` 时，才真正消耗两份硬件资源，在硅片上铺设出两块完全独立、同时工作的采样电路！
2. **父子模块 = 空间包含（大盒子套小盒子），Verilog 没有“类继承”**：
   - 面向对象里的“父子类”是概念上的属性继承（如“猫”继承自“动物”）。
   - 硬件里的“父模块（Parent）”与“子模块（Child）”纯粹是**物理空间上的包含关系**——模块 A 内部例化了模块 B，那么 A 就是包裹在外层的大电路盒（父），B 就是装在 A 肚子里的子电路块（子），就像外层文件夹包含内层子文件夹一样。
3. **工程文件组织规范**：虽然语法允许在一个 `.v` 文件里塞进任意多个 `module`，但**工业界铁律是“一个 `.v` 文件只写一个 `module`，且文件名必须与模块名严格一致”**（如 `half_adder.v` 里只放 `module half_adder`），这样在几十个文件的工程里才能一秒定位源码。

3.2 三级模块分工：叶子模块、中间模块与顶层（Top）

【直觉卡点】顶层一般很大吗？为什么代码看起来这么短？根本逻辑是一级级例化下去吗？

“给我看看叶子模块和中间模块具体长什么样？还有，顶层（Top）一般代码很大吗？为什么顶层只连线不写复杂逻辑？整个设计的根本逻辑就是顶层例化中间层、中间层例化底层，一级一级写下去，最后由顶层和外界交互吗？”

Listing 1: 三级模块层次（叶子 → 中间 → 顶层）完整代码对照

```
// ==================== 1. 叶子模块 (Leaf Module) ====================
// 特点：处于最底层，内部只有 assign / always 逻辑实现，不再例化任何其他模块
module and_gate (
    input  wire a,
    input  wire b,
```

```
    output wire y
);
    assign y = a & b;
endmodule

// ==================== 2. 中间模块 (Intermediate Module) ====================
// 特点：承上启下，把底层的叶子模块当积木拼起来，组合出更高级的功能
module half_adder (
    input  wire a,
    input  wire b,
    output wire sum,
    output wire carry
);
    // 例化叶子模块 u_and，计算进位 carry
    and_gate u_and (
        .a (a),
        .b (b),
        .y (carry)
    );
    // 本层补充异或逻辑，计算本位和 sum
    assign sum = a ^ b;
endmodule

// ==================== 3. 顶层模块 (Top Module) ====================
// 特点：整个工程唯一的最高外壳，端口直接对应芯片物理焊球 (Pin)，内部几乎只做例化与互连
module top (
    input  wire key_a,     // 对应开发板按键 A 物理引脚
    input  wire key_b,     // 对应开发板按键 B 物理引脚
    output wire led_sum,   // 对应开发板 LED0 物理引脚
    output wire led_carry  // 对应开发板 LED1 物理引脚
);
    // 顶层只负责把外界引脚接进核心子系统
    half_adder u_half_adder (
        .a     (key_a),
        .b     (key_b),
        .sum   (led_sum),
        .carry (led_carry)
    );
endmodule
```

【核心结论】三级模块分工口诀与顶层设计哲学

- **三句话分工口诀：叶子只写逻辑，中间只搭积木，顶层只接外界。**
- **为什么顶层（top）代码极短，却代表最大的电路？**就像整栋摩天大楼的“总平面图”只

有薄薄一张纸，上面只画了配电房、电梯井和承重柱的接口位置，而不会在总图上画每一颗螺丝钉。顶层代码越干净（只做子模块例化和总线互连），多人并行开发、模块独立仿真替换、后期排查信号流向就越清晰。

3.3 端口映射的精确语义：.clk(sys_clk) 到底谁连谁？

【直觉卡点】“子模块的 clk 连到父模块的 sys_clk”听起来有方向歧义

“在例化时写 `.clk(sys_clk)`，注释常写‘子模块的 clk 连到父模块的 sys_clk’，这句话到底谁驱动谁？怎么消除歧义？”

【硬核拆解】圆点后永远是“子模块端口”，括号内永远是“父模块信号”

最准确、零歧义的工程表述是：

> **在父模块内部，将子模块实例的端口 `.clk`，焊接到父模块自己的导线 `sys_clk` 上。**

- **语法位置固定**：不论该端口是输入还是输出，**圆点 `.` 后面紧跟的永远是子模块定义时的端口名，括号 `()` 里面填的永远是父模块当前作用域下的信号名。**
- **电信号流向由子模块端口方向决定**：
  - 若子模块的 `clk` 是 `input`：电信号从父模块的 `sys_clk` **流向**子模块的 `.clk`（父驱动子）。
  - 若子模块的 `sum` 是 `output`：电信号从子模块的 `.sum` **流向**父模块的 `led_sum`（子驱动父）。

—

4 第四篇：虚实映射——XDC 物理约束与时序基准

写完所有的 `.v` 文件后，你的电路还只是一团“漂浮在虚拟空间里的逻辑网”，既不知道该连到芯片外壳的哪根金属针脚上，也不知道外面的晶振跳得有多快。这就是 **XDC（Xilinx Design Constraints，设计约束文件）**登场的时刻。

4.1 三级物理传导链路：XDC 绑定的到底是谁？

【直觉卡点】Verilog 里纯粹是变量名，XDC 绑的是顶层还是子模块？自动布线能连对吗？

“模块里纯粹是逻辑变量名，不能直接绑时钟。比如例化写了 `.clk(sys_clk)`，那 XDC 文件里绑定的到底是子模块的 `clk` 还是顶层的 `sys_clk`？配置好 XDC 后，自动布线器就一定能连接对吗？”

从电路板物理焊球到子模块内部的完整传导链路

PCB 板载 50MHz 晶振 ⟶ FPGA 芯片物理引脚（如 W5 焊球）

⇓ **【第 1 棒：由 .xdc 物理约束文件负责绑定】**

顶层模块（top）的对外端口 sys_clk

⇓ **【第 2 棒：由 top.v 内部的例化语句 .clk(sys_clk) 负责连线】**

底层子模块（u_adc）的内部端口 clk

【硬核拆解】逻辑归 Verilog，物理坐标归 XDC，拉线归实现工具

1. **引脚约束只认顶层端口（Top-level Ports）**：只有顶层模块 top 的端口才直接暴露在芯片最外层，因此 XDC 里的 [get_ports ...] **永远填写顶层模块 top 的端口名（sys_clk）**，绝不写子模块内部的 clk。
2. **为什么不在 .v 代码里直接把引脚号写死？**为了**硬件解耦与跨平台复用**。同一个 UART 或滤波器 Verilog 模块，今天用在 Zynq-7020 开发板上，明天迁移到 Kintex-7 板子上，内部 .v 源码一行都不用改，只需换一份几十行的 .xdc 引脚映射表即可。
3. **自动布线器（Router）怎么保证连对？有什么风险？**
   - 只要你的 Verilog 逻辑连接写对、XDC 引脚号和电平标准（如 LVCMOS33）对照开发板原理图填对，Vivado 的自动布局布线器就 **100% 能把内部金属线准确拉通**。
   - **致命警惕**：如果你把 XDC 里的引脚号抄错了（比如把 LED 的引脚号错填成了某个高速时钟引脚），**Vivado 语法检查不会报错**，它会忠实地把信号连到错的引脚上，导致上板后功能失常甚至电平冲突烧毁 IO！所以写 XDC 必须逐字核对原理图。

4.2 逐词拆解 XDC 语法：[get_ports] 与 create_clock

看一段每个工程必写的标准 XDC 约束代码：

Listing 2: 标准引脚绑定与时钟周期约束 (top.xdc)

```
# 1. 绑定物理引脚位置 (PACKAGE_PIN) 与 IO 电压标准 (IOSTANDARD 3.3V)
set_property PACKAGE_PIN W5 [get_ports sys_clk]
set_property IOSTANDARD LVCMOS33 [get_ports sys_clk]

# 2. 声明该引脚输入的时钟频率：周期 20.000ns (即 50MHz)
create_clock -period 20.000 -name sys_clk_pin [get_ports sys_clk]
```

【直觉卡点】看不懂 [get_ports sys_clk] 和 create_clock 的语法构成

“[get_ports sys_clk] 说是嵌套命令、返回端口对象，到底啥意思？还有 create_clock -period 20.000 -name sys_clk [get_ports sys_clk] 里面，哪些是固定关键字，哪些是我们自己定义的变量？”

表 4: create_clock 约束语句逐词语法属性拆解

语句片段	语法身份	具体含义与可否自定义
create_clock	Tcl 核心命令（固定关键字）	告诉时序引擎：我要在设计里定义一个主时钟基准
-period	命令选项参数（固定关键字）	指定接下来要填的数字是“时钟周期（单位默认纳秒 ns）”
20.000	用户传入的数值（根据板子晶振填）	周期 20 ns $\Longleftrightarrow$ 频率 $f = \frac{1000}{20} = 50$ MHz
-name	命令选项参数（固定关键字）	为这个时钟约束起一个在时序报告里显示的别名（可省略）
sys_clk_pin	用户自定义字符串	时钟别名，随便起，方便在时序报告里一眼认出是哪个时钟
[get_ports sys_clk]	Tcl 命令替换（返回数据库对象）	去顶层模块里查找名叫 sys_clk 的端口对象，把时钟绑在它身上

【硬核拆解】用 C 指针与 Python 字典理解 [get_ports sys_clk]

XDC 文件底层其实就是受限的 **Tcl 脚本语言**：

- **方括号 [...] 代表“先执行括号里的命令，拿返回值替换到原位置”**（等同于 Linux Shell 里的 $(...)）。
- **为什么不能直接写字符串 "sys_clk"，非要写 [get_ports sys_clk]？**因为 Vivado 内部维护着一个庞大的硬件网表数据库（里面有端口 Port、引脚 Pin、网线 Net、逻辑单元 Cell）。如果你只传一个字符串 "sys_clk"，工具不知道你指的是顶层端口还是某根内部导线。
- 调用 get_ports sys_clk，就相当于在 Python 里执行 db.ports["sys_clk"]，或者在 C 语言里取指针 &top.sys_clk——它去数据库里精准定位到顶层的 sys_clk 端口，并返回一个**内存对象句柄（Object Reference）**交给外层的 set_property 或 create_clock 去操作。

—

5 第五篇：【核心补充】写完 .v 和 .xdc 之后——从仿真验证到板级闭环

很多初学者写完 .v 源码和 .xdc 约束后，第一反应就是直接点 Generate Bitstream 烧进板子看现象。但在真实的 FPGA 工程中，**“不经仿真和时序检查直接上板”等于蒙着眼睛拆炸弹**——一旦板子没反应，你根本分不清是代码逻辑写错了、复位极性反了、时钟没跑稳，还是外部硬件虚焊了。

写完 .v 和 .xdc 之后，必须依次闯过以下**四大标准工程关卡**，才能真正实现板级闭环。

写完 .v 和 .xdc 之后的工业级标准推进路线

关卡 1：行为级仿真（Testbench） ⟶ 在 PC 内存里注入虚拟激励，看波形验证纯逻辑对不对

⇓

关卡 2：综合（Synthesis）与实现（Implementation） ⟶ 翻译为 LUT/FF 网表，完成物理布局与自动布线

⇓

关卡 3：核查时序报告（Timing Report / WNS） ⟶ 确认满足建立/保持时间（WNS ≥ 0），保证高速不翻车

⇓

关卡 4：生成比特流与片上逻辑分析仪（ILA）抓波 ⟶ 烧录上板，用片内“数字示波器”抓取真实硬件信号

5.1 第一关：行为级仿真（Testbench）——为什么绝不能跳过仿真直接上板？

5.1.1 疑惑一：Testbench 也是 .v 文件，它和我们写的电路模块有什么区别？

【直觉卡点】Testbench 也要烧进板子吗？为什么它没有 input/output 端口？

“听说仿真要写一个叫 Testbench（如 tb_top.v）的文件。它也是 Verilog 写的，那它会不会被综合成电路烧进 FPGA？为什么它的 module tb_top(); 括号里是空的，一个端口都没有？”

【硬核拆解】Testbench 是“虚拟实验室工作台”：左手信号发生器，右手示波器

1. **Testbench 绝对不进板子（不可综合）**：它只在 PC 上的仿真器（如 Vivado XSim、ModelSim、Verilator）里运行。
2. **为什么没有对外端口？**因为你写的待测电路模块（DUT，Design Under Test，比如 top）已经被包在了 tb_top 的肚子里。在 tb_top 内部：
   - 你用 reg 定义虚拟信号源（扮演**虚拟信号发生器**），直接连到 DUT 的 input 端口上；
   - 你用 wire 引出 DUT 的 output 端口（连接到**虚拟示波器**窗口观察波形）。

整个实验台自成闭环，不需要再跟外界连线，所以 `module tb_top();` 端口列表为空！

3. **为什么仿真比上板快 100 倍？**编译一次综合布线动辄需要 5~30 分钟，而且上板后看不到模块内部的中间变量；而跑一次行为级仿真只需 **3 秒钟**，且模块内部每一根 `wire`、每一个 `reg` 在每一拍的变化都能在波形图上一览无余。

5.1.2 疑惑二：怎么写一个最简万用 Testbench？怎么看懂仿真波形里的红线和蓝线？

Listing 3: 最简万用 Testbench 模板 (`tb_top.v`)——可直接套用

```
`timescale 1ns / 1ps  // 时间单位 1ns，时间精度 1ps

module tb_top();      // 注意：Testbench 顶层没有对外端口！

    // 1. 声明驱动 DUT 输入的信号（在 initial/always 里赋值，所以用 reg）
    reg  sys_clk;
    reg  rst_n;
    reg  din;
    // 2. 声明观察 DUT 输出的导线（被子模块驱动，所以用 wire）
    wire dout;

    // 3. 产生 50MHz 虚拟时钟（周期 20ns，即每过 10ns 翻转一次电平）
    initial sys_clk = 1'b0;
    always #10 sys_clk = ~sys_clk;

    // 4. 编写测试激励剧本（initial 块只从第 0 秒开始顺序跑一次）
    initial begin
        // 初始状态：按下复位键（低电平有效），输入置 0
        rst_n = 1'b0;
        din   = 1'b0;
        #100;              // 保持复位 100ns（5 个时钟周期）

        rst_n = 1'b1;      // 松开复位键，电路开始正常工作
        #20  din = 1'b1;   // 过 20ns 给一个高电平激励
        #40  din = 1'b0;   // 再过 40ns 拉低

        #200;              // 继续观察 200ns
        $finish;           // 结束仿真
    end

    // 5. 例化被测模块（DUT）
    top u_dut (
        .sys_clk (sys_clk),
        .rst_n   (rst_n),
```

```
35          .din      (din),
36          .dout     (dout)
37      );
38
39  endmodule
```

【核心结论】看懂仿真波形图的四种颜色状态（四值逻辑：0, 1, X, Z）

打开仿真波形窗口后，除了正常的绿色高低电平（0 和 1），新手一定会遇到另外两种颜色：

- **红线（X，Unknown 未知态）**：最常见的 Bug 信号！原因通常有两个：① **寄存器没有复位初始化**（默认上电就是 X，而且 X + 1 算出来还是 X，会像传染病一样把后续所有电路染红）；② **多驱动冲突**（两个模块同时往一根 wire 上写不同电平）。
- **蓝线（Z，High-Impedance 高阻悬空态）**：代表这根 wire **根本没有接任何驱动源**（比如例化时把信号名拼错了，导致导线悬空）。

5.2 第二关：综合（Synthesis）与实现（Implementation）——工具在后台干了什么？

当仿真波形全绿、逻辑完全符合预期后，才轮到 Vivado 把代码变成物理电路。这个过程严格分为两步：

【硬核拆解】综合 vs 实现：一个管“翻译图纸”，一个管“施工摆放到硅片”

1. **第一步：综合（Synthesis）——从 Verilog 源码到门级网表（Netlist）**
   - **干了什么**：把你的 always、assign、+、* 翻译成 Xilinx 芯片底层真正拥有的标准硬件原语——**LUT（查找表）**、**FDRE（触发器）**、**CARRY4（进位链）**、**RAMB36（块 RAM）**、**DSP48E1（硬件乘加器）**，并生成一张连接关系图（网表）。
   - **这一步抓什么错**：语法错误、多驱动冲突（Multi-driven net）、未连接的必要端口、组合逻辑环路（Combinational Loop）。
2. **第二步：实现（Implementation = Place & Route）——布局与布线**
   - **布局（Placement）**：芯片里有几万个 LUT 和触发器格子，布局器根据 XDC 绑定的引脚位置，决定把网表里的每一个 LUT 和触发器塞进芯片哪个具体的 (X, Y) 物理坐标格子里。
   - **布线（Routing）**：操控芯片内部的金属走线开关矩阵，把摆放好的各个格子用真实的物理铜线连起来，并计算出电信号在每根铜线上跑过所需的真实物理延迟（精确到皮秒 ps）。

5.3 第三关：看懂时序报告与 WNS——怎么知道电路在高速下能不能跑稳？

【直觉卡点】为什么仿真完全正确，写进板子跑 100MHz 却隔三差五算错数？

“我在行为级仿真里波形完美无缺，综合实现也没报 Error，为什么烧进板子后，有时候工作正常，手摸一下芯片或者跑一会儿数据就全乱了?”

【硬核拆解】建立时间（Setup）、保持时间（Hold）与 WNS 指标

行为级仿真属于“理想零延迟仿真”（假设电信号穿过一万个逻辑门也只需要 0 秒）。但在真实硅片上，电信号穿过每个 LUT 和金属线都要花时间！

1. **物理触发器的两条死规矩**：把两个触发器之间的传输想象成“赶高铁检票（时钟上升沿就是停止检票瞬间）”：
   - **建立时间（Setup Time, T_{su}）**：在时钟上升沿到来**之前**，数据必须提前到达下一个触发器门口并稳定站好（不能压哨冲刺）。
   - **保持时间（Hold Time, T_h）**：在时钟上升沿到来**之后**，数据还必须在原地保持稳定一小会儿（不能刚打铃就撤走），否则触发器会抓到半高不低的电压，进入恐怖的**亚稳态（Metastability）**！
2. **为什么组合逻辑写太长会违例？**假设你在 XDC 里约束了时钟周期为 10 ns（100MHz）。如果两个触发器之间连了极其复杂的计算（比如连续做三次 32 位除法再加一堆判断），电信号在中间的 LUT 迷宫里跑一趟需要 13 ns。那么当第 10 ns 的时钟沿打下来时，新数据还在半路上没跑到终点！这就叫**建立时间违例（Setup Violation）**。
3. **工程中怎么一眼判断过没过？只看一个核心指标：WNS！**打开 Vivado 的 `Report Timing Summary`（时序汇总报告），直接看 **WNS（Worst Negative Slack，最差建立时间裕量）**和 **WHS（Worst Hold Slack，最差保持时间裕量）**：

$$\text{Slack（时间裕量）} = \text{时钟允许的最大时间} - \text{信号实际跑完花掉的时间}$$

   - **WNS ≥ 0 且 WHS ≥ 0（黑色数字）**：恭喜！全芯片最慢的一条路径也能在时钟沿到来前跑完，上板绝对稳定。
   - **WNS < 0（红色负数，如 `-2.350 ns`）**：说明最慢的那条逻辑路径超时了 2.35 ns！此时必须修改 Verilog 代码，**在长组合逻辑中间切一刀，插入一级触发器（即打拍流水线 Pipelining）**，把一拍干完的活拆成两拍干完。

5.4 第四关：片上逻辑分析仪（ILA）——上板后内部信号不对怎么抓？

【直觉卡点】芯片内部几千根 wire 没引到外部排针，上板出了问题怎么用示波器测？

“STM32 调试可以打断点单步走，但 FPGA 电路一旦跑起来根本不能暂停（时钟一直在跳）。而且芯片内部子模块之间的信号根本没连到外部排针上，就算有示波器也插不进去测，这怎么调试?”

【硬核拆解】ILA（Integrated Logic Analyzer）：塞进 FPGA 肚子里的“数字示波器”

1. **ILA 的工作原理**：因为芯片内部通常有富余的逻辑资源和块内存（BRAM），我们可以在 Vivado 里调用一个官方 IP 核叫 **ILA**（或者在代码里需要观察的信号前加上标记 `(* mark_debug = "true" *)`）。
2. **实机抓波过程**：
   - 编译时，ILA 电路会和你写的电路一起生成到 `.bit` 比特流（同时生成一个配对的符号表文件 `.ltx`）并烧进 FPGA。
   - 上板运行后，你可以像操作真实示波器一样，在 PC 端设置触发条件（例如：当内部信号 `adc_valid == 1'b1` 的上升沿瞬间触发）。
   - 一旦触发，片内的 ILA 就会以芯片真实的时钟速度（如 100MHz），把触发前后几千拍的内部真实信号值存进片内 BRAM，然后通过 JTAG 线慢速回传到你电脑屏幕上画出波形！

—

6 第六篇：构建链路与底层烧录机制

跑通了代码编写、约束、仿真、实现与调试闭环后，我们站在全局高度，把从源码到烧录文件的全链路及底层物理机制彻底打通。

6.1 为什么只写 .v 不能像软件一样直接“一键生成可执行文件”？

【直觉卡点】理论上不是只需要写 .v，剩下的一键自动生成吗？

“在写 C 语言时，写完一个 `main.c` 点一下编译就能出 `.exe`。为什么在 FPGA 里，只丢给 Vivado 一个 `.v` 文件，不能一键生成可以直接烧录的 `.bit` 文件?”

【硬核拆解】“工程搭好后可以一键”，但“只给一个 .v 绝不可能一键”

Vivado 能自动做综合、布局、布线，但它**不会算命，不能猜你的开发板长什么样**。只有同时集齐以下**六大输入要素**，才能通过脚本实现真正的一键构建：

1. **具体器件型号（Part Number）**：如 `xc7z020clg400-2`（不同芯片内部 LUT 数量和封装焊球完全不同）。

2. **完整的 RTL 源文件列表与顶层模块名（Top Name）**：明确谁是总外壳 `top`。
3. **物理引脚约束（XDC Pinout）**：明确顶层每个端口接在芯片哪个焊球、用多少伏电压。
4. **时钟周期约束（XDC Timing）**：明确外部晶振是多少 MHz，供布局布线器做时序收敛计算。
5. **IP 核与 Block Design（BD）配置**：如 PLL 锁相环、FIFO、Zynq PS 硬核的参数配置。
6. **自动化构建脚本（Tcl / Makefile）**：把上述要素串联起来的施工指令单。

6.2 三条核心构建链路与烧录产物矩阵

在 Zynq 工程中，PL 逻辑、PS 软件以及最终固化到 Flash/SD 卡的整包镜像，遵循三条截然不同的构建链路：

PL、PS 与 Zynq 整包三大构建链路全景图

1. PL 硬件电路构建链路：

$$\texttt{rtl/*.v} + \texttt{xdc/*.xdc} + \texttt{bd/*.tcl} \xrightarrow{\text{综合 Synthesis}} \text{门级网表} \xrightarrow{\text{实现 Implementation}} \text{物理布局布线} \xrightarrow{\texttt{write_bitstream}} \texttt{.bit} \xrightarrow{\texttt{write_cfgmem}} \texttt{.bin}$$

2. PS 嵌入式软件构建链路：

$$\texttt{ps/src/*.c} \xrightarrow{\texttt{arm-none-eabi-gcc -c}} \texttt{*.o}(\text{目标文件}) \xrightarrow{\text{链接器 Linker} + \texttt{lscript.ld}} \texttt{.elf} \xrightarrow{\texttt{objcopy -O binary}} \texttt{.bin}$$

3. Zynq 上电自启动整包打包链路：

$$\texttt{FSBL.elf}(\text{第一阶段引导}) + \texttt{system.bit}(\text{PL 电路}) + \texttt{app.elf}(\text{PS 裸机程序}) \xrightarrow{\text{bootgen 打包}} \texttt{BOOT.bin}$$

6.3 .bit 比特流解剖学、JTAG 位序与主被动加载模型

【直觉卡点】.bit 是固定偏移扁平表吗？JTAG 位序和内存一样吗？到底谁推谁读？

“.bit 文件是不是规定了前多少位固定代表什么、后多少位代表什么的扁平表？JTAG 一位一位移入芯片的顺序，和文件在内存里的 bit 顺序一样吗？烧录时到底是工具推还是板子自己读？”

【硬核拆解】协议流结构、LSB/MSB 位反转与三种硬件加载拓扑

1. **.bit 不是死板的固定偏移扁平表，而是“ASCII 文件头 + 分层配置协议包流”**：
   - **文件最开头（可变长 ASCII 头部）**：记录了顶层模块名、目标芯片型号（防烧错芯片）、编译日期和时间。
   - **核心载荷（二进制配置协议指令流）**：紧接着是一串专门给 FPGA 内部配置状态机解析的指令包——先发一串特殊的**同步字（Sync Word：0xAA995566）**唤醒配置

表 5: 不同调试与固化场景下的烧录文件选型表

应用场景	使用的文件格式	断电是否丢失	底层原理解释
JTAG 在线调试 PL	`.bit`（配 `.ltx`）	**断电即丢**	直接写入 FPGA 内部基于 SRAM 的配置单元，速度最快，适合开发阶段反复修改调试
JTAG 在线调试 PS	`.elf`	**断电即丢**	包含机器码、内存段地址表与调试符号表，由调试器直接灌入板载 DDR 内存运行
纯 FPGA 固化到 Flash	`.bin` / `.mcs`	**断电保存**	剥离了 `.bit` 开头的 ASCII 信息头，纯二进制数据，烧进片外 SPI Flash 供上电自启
Zynq 整包固化（SD/Flash）	`BOOT.bin`	**断电保存**	Zynq 上电后先由硬核 BootROM 读取 `FSBL.elf`，再由 FSBL 自动加载 `.bit` 配置 PL 并启动 `app.elf`

器 → 发送**写寄存器命令** → 指定**帧地址（Frame Address）** → 灌入海量**帧配置数据（LUT 真值表与布线开关状态）** → 校验 **CRC 校验码**（若传输错一位立即拒绝启动，防止内部短路烧芯片）→ 最后发送 **START 启动命令**释放全局复位。

2. **内存 bit 顺序 ≠ JTAG 物理线上的移位顺序**：虽然按字节（Byte）看宏观先后顺序一致，但在每个字节内部的 8 个 bit 上，JTAG 移位寄存器通常要求 **LSB-first（最低有效位先移入）**或特定的位反转（Bit-swapping）。而且 JTAG 软件工具在下载 `.bit` 时，会自动剥离掉开头的 ASCII 头部，只提取同步字之后的纯配置流并完成位序转换后，再推入芯片。

3. **到底是谁在推、谁在读？（三种加载模式边界）**

   - **JTAG 在线下载（PC 主动推，FPGA 被动收）**：电脑上的 Vivado 通过下载器控制 TCK 时钟，强行把 bit 流一位位塞进 FPGA。
   - **纯 FPGA 从 Flash 启动（FPGA 主动读，Flash 被动吐）**：板子一通电，FPGA 内部硬连线的配置电路充当 SPI Master，主动发出时钟去片外 Flash 里把 `.bin` 读进来配置自己。
   - **Zynq SoC 启动（PS 接管一切，通过 PCAP 给 PL 灌配置）**：Zynq 上电永远是 **PS（ARM 核）先醒**！ARM 核先从 SD 卡或 Flash 读出 `BOOT.bin`，然后通过芯片内部的专属通道 **PCAP（Processor Configuration Access Port）**把里面的 `.bit` 数据灌给旁边的 PL，帮 PL 完成电路搭建。

—

7 第七篇：工业级工程规范与目录架构

当全部技术脉络跑通后，我们回到工程管理层：为什么一个规范的工业级 Zynq/FPGA 仓库要这样设计目录？哪些文件必须当作资产保护，哪些可以随时清空？

7.1 标准工程目录树与三端职责隔离

工业级 Zynq 异构工程标准目录结构

```
project_root/
   ├ fpga/        #【PL 侧】只放硬件电路设计：rtl/*.v、xdc/*.xdc、bd/*.tcl、scripts/*.tcl
   ├ ps/          # 【PS 侧】只放片上 ARM 软件：src/*.c、include/*.h、链接脚本 lscript.ld
   ├ host/        # 【PC 侧】只放电脑上位机程序：Python 串口/网口控制脚本、波形分析工具
   ├ build/       # 【临时工作区】Vivado/GCC 编译产生的海量中间缓存、日志（可随时删除重建）
   ├ artifacts/   #【交付物归档】经过验证的最终黄金产物：top.bit、app.elf、BOOT.bin
```

```
（严禁乱删）
    ├ evidence/     # 【验证证据链】与 artifacts 一一对应的时序报告、资源报表、实测波形截图
    └ Makefile      # 【总控入口】顶层一键构建、清理、归档规则文件
```

7.2 生成物生命周期：build/（厨房半成品）vs artifacts/（留样成品）

【直觉卡点】build/ 和 artifacts/ 都是编译生成的，为什么一个能删、一个必须保存？

“build/ 真的可以删除吗？它主要放什么？既然 artifacts/ 也是构建出来的产物，为什么不能删掉需要时再重新编译？”

表 6: build/ 与 artifacts/ 核心工程属性对比

评估维度	build/（构建工作区 / 厨房操作台）	artifacts/（最终交付物 / 出锅留样菜品）
存放内容	Vivado 临时工程（.xpr）、数以千计的综合/实现中间缓存、.o 目标文件、过程日志	最终定版的 system.bit、调试符号表 system.ltx、app.elf、BOOT.bin
体积大小	极其庞大（动辄数 GB），不进 Git 仓库（写入 .gitignore）	精简小巧（几 MB 到几十 MB），按版本号严格归档保存
可重复性风险	随时可通过脚本从源码重新生成	**受操作系统、Vivado 小版本号、布局布线随机种子影响**，半年后换台电脑重跑出的 .bit 物理走线可能不同
删除策略	**可安全删除**（前提：源码、XDC、BD Tcl 脚本齐全，且黄金产物已归档至 artifacts/）	**严禁随意删除！**它是与实验测试报告（evidence/）唯一对应的基准镜像

7.3 自动化构建体系：scripts/、Makefile 与词源辨析

【直觉卡点】fpga/scripts/ 和嵌入式里的 Makefile 一样吗？scripts 是不是 JavaScript？

“fpga/scripts/ 构建脚本英文是什么？它和嵌入式里的 Makefile 一样吗？还有，scripts 是脚本的意思吗？和 JavaScript 有什么关系？”

【硬核拆解】声明式总管（Makefile）+ 命令式施工员（Tcl Scripts）

1. **词源破除乌龙**：
   - scripts 就是英文单词 script（脚本/剧本）的复数形式，fpga/scripts/ 英文统称 **Build Scripts（构建脚本目录）**。它只是一个“装脚本文件的抽屉”，里面可以

放 Tcl 脚本（`.tcl`）、Shell 脚本（`.sh`）或 Python 脚本（`.py`）。

- **`JavaScript`** 是前端网页开发用的一门特定编程语言，和硬件工程里的 `scripts/` 目录没有任何关系（也不存在 `JAVAscripts` 这种写法）。

2. **`Makefile` 与 `fpga/scripts/*.tcl` 的协同分工**：两者目标一致（都是为了摆脱鼠标乱点 GUI，实现一键可重复构建），但机制处于不同层级：

   - **`Makefile`（由 `make` 解析的“声明式依赖总管”）**：它站在整个工程最高处，判断“如果 `rtl/top.v` 修改时间比 `build/top.bit` 新，就触发重新构建 PL；如果只改了 `ps/main.c`，就跳过漫长的 FPGA 综合，只花 1 秒钟重编 ARM 的 `.elf`”。
   - **`fpga/scripts/build_pl.tcl`（由 `vivado -mode batch -source` 解析的“命令式施工员”）**：`make` 本身不懂怎么操作 FPGA，当 `Makefile` 决定要重编 PL 时，就会在后台调用 Vivado 去逐行执行 `.tcl` 脚本里的具体命令（`read_verilog` → `synth_design` → `place_design` → `route_design` → `write_bitstream`）。

【核心结论】全篇终极心法总结

从“写程序”跨越到“设计硬件系统”，本质上是掌握四条核心法则：

1. **空间思维：你写下的每一行可综合代码，都是硅片上真实通电并排运转的 LUT 与触发器；**
2. **边界思维：模块靠端口隔绝内外，XDC 靠顶层连接虚实，PL 与 PS 靠总线协同软硬；**
3. **节拍思维：时钟上升沿是全芯片唯一的指挥棒，满足建立与保持时间（WNS ≥ 0）是一切稳定的前提；**
4. **闭环思维：写完代码只是开始，“Testbench 仿真 → 时序收敛核查 → ILA 实机抓波”才是通向工业级可靠性的必由之路。**